Reto 1 · Vivienda joven en Euskadi

Informe técnico de la versión en desarrollo. Orientación basada en rentas municipales observadas; no concede ni deniega ayudas. La entrega integral sigue pendiente de los requisitos no acreditados.

Usuario: servicios de orientación y jóvenes que comparan una referencia territorial con su presupuesto. Objetivo: automatizar referencias trazables, evitar confundir promociones con ofertas y explicar las reglas. El beneficio esperado es facilitar comparaciones; no se ha medido ahorro de tiempo ni impacto social.

Arquitectura: capturas verificadas → preparación → HDFS raw/processed → Kafka → MongoDB → vistas. EMAL → división temporal → pipeline sklearn → modelo conservado → Streamlit → reglas → n8n → ficha local. Los modelos y estadísticas municipales tienen unidades diferentes de un anuncio individual.

Datos y metodología


```
{
  "fuente": {
    "archivo": "emal_municipios.xlsx",
    "url_solicitada": "https://opendata.euskadi.eus/contenidos/estadistica/122417_emal_tabla
s_estad/opendata/EMAL.-Barrios-Municipios.-2016-2026_es.xlsx",
    "fecha_utc": "2026-11-10T14:37:18.098528+00:00",
    "http_status": 200,
    "url_final": "https://opendata.euskadi.eus/contenidos/estadistica/122417_emal_tablas_est
ad/opendata/EMAL.-Barrios-Municipios.-2016-2026_es.xlsx",
    "estado": "descargado",
    "bytes": 256980,
    "sha256": "f4d27d4e8e9bd3688d4e7a454026a4a0884073dc73e3fc1c0da4891a1ad1af5c"
  },
  "unidad": "municipio/trimestre; nuevos contratos EMAL",
  "observaciones": 768,
  "municipios": 19,
  "objetivos_no_publicados_excluidos": 11,
  "exclusion_territorial": "48036 Galdakao: cambio de perimetro",
  "advertencia": "Serie revisada en la fecha de captura; evaluacion retrospectiva, no prueba
de disponibilidad historica en tiempo real. No representa anuncios individuales ni todos los
municipios de Euskadi.",
  "smote": "No aplicable al objetivo continuo de renta; comparacion binaria auxiliar
separada sobre subidas observadas.",
  "particiones": {
    "train": {
      "filas": 601,
      "desde": 8064,
      "hasta": 8095
    },
    "val": {
      "filas": 75,
      "desde": 8096,
      "hasta": 8099
    },
    "test": {
      "filas": 92,
      "desde": 8100,
      "hasta": 8104
    }
  }
}
```


Aprendizaje supervisado de regresión, IA débil especializada en una variable y un territorio. No razona sobre derechos administrativos ni representa expedientes individuales. Pipeline común: imputación, StandardScaler y OneHotEncoder dentro de cada fold temporal. Las entradas son año, trimestre, municipio/provincia y retardos observados. Galdakao se excluye por cambio territorial; los objetivos no publicados no se imputan. La evaluación es retrospectiva sobre una serie revisada.


```
campo,tipo,nulos_pct_train
codigo_municipio,object,0.0
municipio,object,0.0
anio,int64,0.0
trimestre,int64,0.0
renta_media_eur,float64,0.0
numero_contratos,float64,0.0
periodo,int64,0.0
renta_lag1,float64,4.159733777038269
renta_lag4,float64,13.643926788685523
contratos_lag1,float64,2.995008319467554
provincia,object,0.0
```

Comparación extraída de MLflow

Modelo | MAE train | MAE val | RMSE val | R² val | segundos
MLP_2_capas | 15.88 | 24.20 | 31.86 | 0.9370 | 0.080
Random_Forest | 5.92 | 25.15 | 34.76 | 0.9250 | 1.846
MLP_regularizada | 14.78 | 25.42 | 34.03 | 0.9282 | 0.125
MLP_base | 14.22 | 25.78 | 34.38 | 0.9267 | 0.126
XGBoost | 11.64 | 26.52 | 37.05 | 0.9148 | 0.249
Regresion_lineal | 12.76 | 31.17 | 40.58 | 0.8978 | 0.226
MLP_tanh | 16.97 | 35.16 | 50.79 | 0.8400 | 0.139
SVM | 10.60 | 37.86 | 55.56 | 0.8085 | 0.186
KNN | 0.00 | 51.52 | 60.89 | 0.7700 | 0.092

Ganador conservado: MLP_2_capas. MAE de validación: 24.20 EUR; persistencia: 24.63 EUR. La mejora es pequeña; no demuestra superioridad generalizable. No se reselecciona después de observar test. MAE mide error absoluto medio en euros; RMSE penaliza más los errores grandes. R² compara con la media, no es porcentaje de aciertos.

El MLP usa sklearn (base y tres experimentos), no Keras. El enunciado pide MLP sin fijar biblioteca; los ejercicios adjuntos usan Sequential/Dense/Adam y se documentan como diferencia docente. SMOTE no corresponde a regresión; el experimento auxiliar clasifica subidas trimestrales observadas. SHAP utiliza validación y fondo de train, con interpretación no causal.

Evaluación final conservada


```
{
  "modelo": "MLP_2_capas",
  "n_test": 92,
  "mae": 28.66563646053045,
  "rmse": 37.9285330648495,
  "r2": 0.918932902983818,
  "sha256_modelo": "d6f05d11fa6002de53ecaff671b697bd901f63f946b87cc7484701aea76c210a",
  "sha256_test": "f1520fc0580fee119b031252538961e5d62eee68db6a0783c3a5384bc5a65011",
  "evaluaciones": 1,
  "aviso": "Holdout retrospectivo de agregados EMAL; no valida elegibilidad ni decisiones
personales."
}
```


Esta generación solo lee el informe y la figura existentes. No importa joblib, no ejecuta inferencia sobre test ni ajusta hiperparámetros. Las reaperturas de la app reutilizan el informe sellado.

Resultados municipales y límites

```
# Estado de entrega y comprobación por asignatura

## Actualización · 12 de noviembre de 2026

La captura de anuncios está acreditada con **17 identidades de dos portales**
(9 Habitaclia y 8 Pisos.com), conservadas con robots, HTML y SHA-256 en
`data/capturas_anuncios/2026-11-12/`. Se han transportado por HDFS → Kafka → MongoDB
junto con 621 registros de promociones: **638 registros**, sin confundir sus unidades.
La muestra cubre las tres capitales y mezcla condiciones de alquiler; no es
representativa ni acredita disponibilidad o uso habitual.

Resultados, consultas, índices y límites: [RESULTADOS_ANUNCIOS.md](RESULTADOS_ANUNCIOS.md).
El visor territorial incorpora la muestra archivada, precios y superficies.
Las menciones al bloqueo de red del 11 de noviembre que siguen son históricas;
en esta máquina ambos robots responden HTTP 200 y la API GitHub funciona.
Validación local final: **159 pruebas y 12 integraciones reales**, sin omisiones.
El código a623ece pasó el CI BDA 43242707927 (incluido el lote archivado),
MongoDB 43242707429 y Windows/Linux 43242707545, con ambas aplicaciones.
Se integró normalmente en test/main y se volvió a dev-jon; ver CONTINUAR.md.
El modelo y su test único permanecen intactos. No se han usado los anuncios
para entrenar ni reevaluar el holdout.


## Actualización del 11 de noviembre de 2026

La adquisición de anuncios tiene ahora un CLI trazable y comprobaciones de
robots/TLS/originales antes de ingerir. Sus 153 pruebas incluyen fixtures de
HTML y fallos de acceso; las cinco pruebas de servicios se repitieron sin skips.
El intento real produjo cero anuncios por CONNECT 403. Consultar
[captura y límites](SCRAPING.md) y [evidencias](evidencias_scraping_2026-11-11/).
Los selectores no se declaran validados con páginas actuales inaccesibles.
El [CI BDA 43011875465](https://github.com/markelalvarez0/Reto-1/actions/runs/43011875465)
terminó Success para `a15756c`, sin convertir las pruebas con fixtures en una
captura vigente de mercado.

Se conserva la decisión del modelo y el test único. El incremento anterior aportó
129 pruebas unitarias, siete pruebas de MongoDB real y cinco pruebas de
HDFS/Kafka/MongoDB sin omisiones. Se transportaron tanto 621 promociones
oficiales como cinco vistas municipales verificadas. Las ejecuciones no son
equivalentes a un scraping vigente de alquileres. Ver
[evidencias locales](evidencias_2026-11-11/) y [contraste docente](CONTRASTE_DOCENTE.md).

El informe corregido se genera con `python main.py --pdf`; lee métricas,
diccionario, figuras y documentos verificados, sin deserializar el modelo.
Produce Markdown, PDF paginado y manifiesto de fuentes en `data/informe_entrega/`.
Los generadores históricos ahora delegan en él. El PDF maestro adjunto original
es contexto histórico y no debe confundirse con el informe vigente.

CI remoto del incremento **1c7c45e**: [BDA
43008668473](https://github.com/markelalvarez0/Reto-1/actions/runs/43008668473)
y [n8n 43008668600](https://github.com/markelalvarez0/Reto-1/actions/runs/43008668600),
ambos **Success**. Los cuatro PDF versionados contienen ahora el informe corregido
paginado. No se confunden con el PDF histórico adjunto de la conversación.

## Objetivo confirmado

Estimar alquileres con datos observados y orientar al joven mediante reglas. El
modelo conservado estima **renta media municipal trimestral**, no el precio de un
piso ni la solvencia individual. Su selección y evaluación final están cerradas;
abrir la aplicación o ejecutar la auditoría no vuelve a entrenar ni evalúa test.

## Incremento de desarrollo: acceso, integridad y comparación territorial
```

Resultados municipales y límites (continuación)

- `main.py` abre Streamlit y el modelo conservado; conserva los códigos de error,
  ejecuta la suite completa y resuelve rutas desde la raíz del proyecto.
- `python main.py --audit --json` verifica los seis originales, los 88 artefactos
  conservados, el informe de test único y las tres copias académicas de `src/`.
  No carga archivos pickle/joblib ni ejecuta predicciones.
- `scripts/analyze_municipal.py` carga estadísticas EMAL en `rentas_emal` y el
  contexto OVJ en `contextos_ovj`, conservando las huellas. La identidad incluye
  captura, INE y trimestre. La recarga no duplica documentos ni borra colecciones.
- Tres agregaciones MongoDB producen comparación municipal, serie temporal y
  resumen territorial. El `$lookup` cruza cada observación con el contexto
  autonómico documentado; nunca lo etiqueta como salario municipal.
- `app/territorio.py` lee salidas con manifiesto SHA-256, muestra tablas, gráficos,
  CSV descargable y coordenadas de promociones Visesa. La cartografía base usa
  recursos externos; su disponibilidad no está garantizada en una red restringida.

Cómo ejecutarlo

Python 3.12. Desde la raíz:

```bash
uv sync --locked --extra dev
uv run --no-sync python main.py --audit --json
uv run --no-sync python main.py --subject bda_sbd --prepare-only

# MongoDB local accesible; MONGO_URI permite cambiar el servidor/puerto.
uv run --no-sync python scripts/analyze_municipal.py
uv run --no-sync python main.py --subject bda_sbd --visual
uv run --no-sync python main.py --subject saa_pia_mia

uv run --no-sync python main.py --test -q
# Con MongoDB real configurado en MONGO_INTEGRATION_URI:
uv run --no-sync python -m pytest integration_tests/test_mongodb_live.py
integration_tests/test_municipal_live.py -q
```

Las salidas regenerables se guardan en `data/analisis_municipal/`, fuera de Git.
`--simulate-db` permite desarrollo sin servidor y marca el motor como mongomock;
no es evidencia de ejecución en MongoDB real. Se mantiene la alternativa pip
documentada en el README. Las versiones del lock actual no sustituyen el registro
histórico del entorno con el que se entrenó el modelo publicado.

Resultado descriptivo reproducido: 2024 T4

18 municipios con renta publicada en ese trimestre. La serie completa conservada
tiene 19 municipios; no se rellena el dato ausente para completar la tabla.

| Municipio | Renta media mensual | Contratos | Variación interanual | Esfuerzo del
escenario |
|---|---:|---:|---:|---:|
| Vitoria-Gasteiz | 798,57 € | 549 | 2,88 % | 51,06 % |
| Bilbao | 930,31 € | 767 | 6,86 % | 59,48 % |
| Donostia-San Sebastián | 1.147,53 € | 516 | 8,53 % | 73,37 % |

El escenario utiliza 1.564 €/mes, salario neto medio de juventud asalariada de
18–34 años en Euskadi en 2024, incluidas pagas extra (OVJ). Entre estas tres
capitales, la renta media de Vitoria es menor, pero las tres superarían el 30 %
de ese ingreso si una persona pagara íntegramente la renta media municipal.
Esto ilustra una dificultad económica; no demuestra qué proporción de jóvenes
puede emanciparse ni representa todos los hogares o contratos actuales.

Los indicadores son renta mensual, equivalente anual, número de contratos,

Resultados municipales y límites (continuación)

variación interanual cuando existe el dato previo, esfuerzo del escenario y
diferencia frente al umbral analítico del 30 %. Este umbral no concede ni deniega
subvenciones. Las medias territoriales son simples entre municipios cubiertos,
no medias de todos los contratos de cada provincia.

Rúbricas: evidencia y trabajo que permanece

Asignatura	Evidencia utilizable	Pendiente para cerrar toda la entrega
BDA	Capturas oficiales y vistas municipales verificadas en HDFS/Kafka/MongoDB reales, con recarga idempotente y CI correcto en 1c7c45e.	Muestra de 17 anuncios archivada el 12 de noviembre y transportada con servicios reales; mezcla de modalidades documentada. Ensayo de defensa pendiente.
SBD	Modelado, limpieza, consultas, agregaciones e índices con mongod real; comparación EMAL/OVJ y transporte de vistas incluidos en evidencias.	Ensayo de la demostración y defensa. Las consultas de anuncios se prueban con fixtures identificados.
SAA	EDA, cinco familias, MLP y experimentos, SHAP y test único conservado. Contraste docente documentado.	Defensa individual. sklearn no se presenta como Keras; no se cambia el ganador mirando test.
PIA	Pipeline, MLflow, notebooks, Streamlit y PDF corregido desde artefactos verificados. Capturas y prueba de reapertura actuales.	Windows y Linux validados para suite, preparación, PDF y ambas apps. Los servicios HDFS/Kafka/MongoDB se validaron en Linux.
MIA	Ocho ramas n8n y recorrido Streamlit → consentimiento → ficha local, con persistencia e idempotencia reales.	Ensayo humano final. Ninguna prueba tramita ayudas ni comunica con personas.

Los 33 tests históricos fueron ampliados en `dev-jon`; contar tests no sustituye
revisar qué prueban. Los avisos de convergencia de mini-tests no son resultados de
entrenamiento de entrega. Los originales, los artefactos ML y el test final
permanecen conservados byte a byte.

Se ha preparado [defensa de 15 minutos y preguntas](DEFENSA.md). Permanece pendiente el
ensayo humano; la captura de anuncios está acreditada en la actualización del 12 de
noviembre.
La integración técnica en test/main se completó tras la validación de a623ece. Las cifras
históricas de la tabla anterior
deben leerse junto a esta actualización, no como trabajo aún sin implementar.

Contratos BDA y evidencia

````
# BDA: ingesta trazable y transporte de las capturas oficiales

## Actualización · 12 de noviembre de 2026

La captura de anuncios está acreditada con **17 identidades de dos portales**
(9 Habitaclia y 8 Pisos.com), conservadas con robots, HTML y SHA-256 en
`data/capturas_anuncios/2026-11-12/`. Se han transportado por HDFS → Kafka → MongoDB
junto con 621 registros de promociones: **638 registros**, sin confundir sus unidades.
La muestra cubre las tres capitales y mezcla condiciones de alquiler; no es
representativa ni acredita disponibilidad o uso habitual.

Resultados, consultas, índices y límites: [RESULTADOS_ANUNCIOS.md](RESULTADOS_ANUNCIOS.md).
El visor territorial incorpora la muestra archivada, precios y superficies.
Las menciones al bloqueo de red del 11 de noviembre que siguen son históricas;
en esta máquina ambos robots responden HTTP 200 y la API GitHub funciona.
El modelo y su test único permanecen intactos. No se han usado los anuncios
para entrenar ni reevaluar el holdout.


## Continuación validada el 11 de noviembre de 2026

En esta máquina se ejecutaron cinco pruebas end-to-end sin skips con Hadoop
3.4.2 lean (SHA512 oficial), Java 11, Kafka 3.9.1 y MongoDB 7.0.
La preparación volvió a aceptar 621 registros (215 Visesa y 406 Etxebide),
rechazando dos de los 623 originales por municipio ambiguo. Llegaron y se
persistieron los 621, con procedencia y recarga idempotente.
Ver [XML](evidencias_2026-11-11/integracion.xml) y
[resumen local](evidencias_2026-11-11/bda-resumen.json).

El análisis territorial se exportó con MongoDB real y se transportaron sus cinco
vistas por separado. `scripts/transport_municipal.py` verifica el manifiesto,
almacena procesados en HDFS, recupera y compara bytes, publica lo recuperado en
Kafka y persiste en `vistas_municipales` antes de confirmar offsets.
La identidad `(archivo, sha256)` permite reentrega sin duplicar. Los mensajes son
vistas agregadas, no HousingAd ni viviendas nuevas. Una nueva huella conserva
otra versión. La prueba repite todo el transporte y comprueba cinco documentos.

```bash
python scripts/analyze_municipal.py --output-dir evidencia_bda_integracion/municipal
python scripts/transport_municipal.py --input-dir evidencia_bda_integracion/municipal
--report evidencia_bda_integracion/transporte_municipal.json
```

MONGO_URI debe seleccionar el mismo servidor para ambos comandos. El workflow
registra MONGO_INTEGRATION_URI para la prueba municipal, exporta las vistas,
ejecuta la auditoría y rechaza XML vacíos, errores, fallos, skips y ausencia de
las pruebas municipales. Archiva los resultados y sus huellas. Esta validación
local precede al commit; no se presenta como un CI remoto de checkout limpio.

## Alcance de esta correccion

Este recorrido usa los originales conservados en `data/capturas_oficiales/`, no anuncios
inventados. Integra los catalogos de promociones de Visesa y Etxebide, el nomenclator INE y
el contexto salarial publicado por el OVJ. **Una promocion no equivale a una vivienda
disponible ni a un anuncio individual de alquiler.**

Se distinguen codigo implementado, pruebas con fixtures y pruebas con capturas reales. La
evidencia de este recorrido, cuando la validacion termina correctamente, queda en
`docs/evidencias_bda/resumen.json` y en el workflow `bda-candidate.yml`.

## Contratos de los datos
````

Contratos BDA y evidencia (continuación)

Fuente	Contrato y tratamiento
Visesa	La captura verificada usa `rows`. El lector conserva esa compatibilidad y admite por separado `promociones`, sin convertir precios de locales o garajes en rentas. Un contrato desconocido provoca un error, no un exito con cero filas.
Etxebide	CSV con `;`, codificacion ISO-8859-1 y columnas `ID_PROM`, `Cod_Prov`, `CODMUNI`, `Municipio` y `Reg Acceso`. No publica una renta mensual individual en esas columnas.
INE	XLSX con hojas provinciales: se leen `01`, `20` y `48`. Se concatenan CPRO (2 digitos) y CMUN (3), sin incluir DC.
OVJ	Se extrae el salario del parrafo correspondiente a 18-34 anos, 2024, neto mensual e incluidas pagas extraordinarias. Se conserva como referencia de Euskadi, no como salario de cada municipio.

Los nombres historicos de Etxebide se resuelven por el codigo publicado, si existe en el catalogo INE verificado y no contradice un nombre reconocido. Se conservan el nombre original y el metodo de correspondencia. No se aceptan coincidencias parciales ni se asigna Bilbao como localidad por defecto.

Integridad de los originales

Se detecto que `* text=auto` normalizaba CRLF a LF al incorporar el CSV de Etxebide y el HTML del OVJ a Git. El contenido textual seguia siendo equivalente, pero la huella de los bytes ya no coincidia con el manifiesto.

La correccion recupera los bytes exactos del artefacto original de captura, sin recapturar la fuente ni modificar las huellas esperadas. La regla `/data/capturas_oficiales/** -text -diff` impide nuevas conversiones. Antes de procesar se verifica SHA-256 de cada original; una diferencia detiene la carga.

Recorrido reproducible

1. Verificar manifiesto, originales e INE; preparar `vivienda_oficial.json` y `municipios_oficiales.json` fuera de la carpeta de originales.
2. Conservar en HDFS tanto las capturas originales como los derivados en rutas `raw` y `processed` separadas. Recuperar cada fichero y comparar bytes y SHA-256.
3. Publicar registros `HousingAd` con fuente, tipo de registro, procedencia y lote de ingesta. Las claves de particionado son codigos municipales.
4. Consumir solo el lote esperado y confirmar el desplazamiento de Kafka despues de la persistencia en MongoDB.
5. Guardar por `(fuente, id_origen)` sin borrar colecciones ni reiniciar contadores. Comprobar que la procedencia sobrevive al transporte.
6. Exportar recuentos de registros de promociones por municipio y fuente. Las estadisticas de alquiler individual excluyen `tipo_registro=promocion`.

Los recuentos de dos catalogos **no deben sumarse como viviendas unicas**: puede haber promociones presentes en ambas fuentes. Los importes y superficies desconocidos siguen siendo nulos. El salario autonomico se guarda en `contexto_autonomico.salario_ovj`; el campo de salario municipal no se rellena con ese valor.

Ejecucion

Desde la raiz del repositorio, con el entorno de `requirements-ci.txt` instalado:

```bash
# Solo comprobar y preparar originales; no requiere servicios.
python "BDA + SBD/main.py" --prepare-only

# Recorrido real: requiere HDFS, Kafka y MongoDB accesibles.
python "BDA + SBD/main.py" --pipeline

# Configuracion explicita del lote y salidas.
```

Contratos BDA y evidencia (continuación)

```
python scripts/run_pipeline.py --snapshot-dir data/capturas_oficiales --raw-dir
data/bda/prepared --output-dir data/bda/processed --db-name reto1_vivienda --interval-
seconds 0.1

# Demostracion separada, con datos y servicios simulados identificados.
python scripts/run_pipeline.py --demo

# Regresion y sincronizacion de codigo.
python -m pytest tests/ -q
python scripts/sync_sources.py --check
```

El comando de la raiz `python main.py --subject bda_sbd --pipeline` delega en el mismo recorrido. No es necesario copiar otro `src/` al PYTHONPATH. Las conexiones usan `MONGO_URI`, `MONGO_DB`, `HDFS_NAMENODE_HOST`, `HDFS_NAMENODE_PORT`, `HDFS_USER`, `KAFKA_BOOTSTRAP_SERVERS` y las opciones del cliente; consultar `.env.example`.

Verificacion y limites pendientes

La validacion usa Hadoop, Apache Kafka y MongoDB temporales en GitHub Actions. Comprueba los originales, el transporte, la persistencia, los recuentos, las uniones municipales y una recarga idempotente. No modifica el modelo ML ni vuelve a evaluar su test reservado.

Esa ejecución histórica no acreditaba anuncios. La muestra del 12 de noviembre sí aporta 17 anuncios; no acredita la descarga de todos los indicadores Udalmap. Los scrapers y clientes antiguos que no intervienen en este recorrido requieren su propia revision, incluido el cliente OVJ heredado con constantes. No deben usarse sus salidas historicas como evidencia observada. La comparación actual emplea EMAL y OVJ verificados, no esas constantes.

El mapa histórico no se abre como si contuviera estos resultados. El visor territorial actual está comprobado con resultados de MongoDB real y muestra promociones localizadas, no ofertas disponibles. El incremento del 12 de noviembre añade indicadores de la muestra de anuncios; no representan el conjunto del mercado libre.

Muestra de anuncios y conclusiones

Muestra de anuncios observada el 12 de noviembre de 2026

Se han conservado 17 identidades de anuncio: 9 de Habitaclia y 8 de Pisos.com.
La captura fue acotada a tres candidatos interpretables por listado y portal;
Pisos.com publicó solo dos interpretables en Vitoria. No es una muestra aleatoria,
un censo, una comprobación de disponibilidad ni 17 viviendas necesariamente distintas.
Los anuncios de portales diferentes pueden corresponder al mismo inmueble.

Adquisición y comprobación

En este entorno la política de red está aplicada en modo sin lista restrictiva.
Los dos robots y los avisos legales se pudieron consultar con HTTPS verificado.
Las rutas de listados están permitidas para `Reto1AcademicBot/1.0`; se respetaron
robots, pausas y redirecciones del mismo dominio. Los avisos reservan derechos
sobre los contenidos; robots no es una licencia. El uso es académico, sin
explotación comercial, contactos con anunciantes ni seguimiento automatizado.

La primera ejecución conservó un fallo interpretable: URLs antiguas 404 y
selectores desactualizados. Después se corrigieron las rutas de Habitaclia y
ambos parsers usando el HTML recibido. El lote final es
`data/capturas_anuncios/2026-11-12/`; sus 22 ficheros incluyen informe, anuncios,
robots, listados y metadatos. `data/capturas_anuncios/manifest.json` sella sus
bytes. `.gitattributes` impide normalizarlos en Windows.

La comprobación offline vuelve a extraer los campos desde el HTML archivado y
contrasta identidad, URL, precio, municipio INE, superficie, habitaciones y baños
con los documentos de MongoDB. No requiere repetir peticiones a los portales.
El detalle de tarjetas no interpretables y del límite de selección consta en
`evidencias_entrega_2026-11-12/anuncios/resumen.json`: no se confunden los
candidatos no seleccionados por el límite con registros rechazados en transporte.

Indicadores descriptivos de la muestra

Municipio	Anuncios	Precio medio pedido €/mes	Mínimo	Máximo	Superficie media m²	Media de €/m²
Vitoria-Gasteiz	5	1.604,40	750	3.150	95,80	16,60
Donostia-San Sebastián	6	2.108,33	1.350	2.500	86,00	25,70
Bilbao	6	2.316,67	1.200	4.500	105,50	21,36

El precio medio pedido de esta selección es mayor en Bilbao; la media de €/m²,
mayor en Donostia. Son composiciones diferentes de viviendas y modalidades:
estas diferencias no prueban un orden general de precios entre municipios.
La media de €/m² se calcula sobre las razones de cada anuncio, no dividiendo
las dos medias de la tabla. Los precios están publicados en mensualidades.

Se detectan tres anuncios con señales textuales de temporalidad y dos dirigidos
a empresas; las categorías pueden solaparse. La ausencia de esas palabras no
acredita que un alquiler admita vivienda habitual. No hay nulos de precio,
superficie, habitaciones o baños entre los seleccionados. Las nueve tarjetas de
Habitaclia carecen de descripción; se conserva nula. Ascensor permanece nulo en
los campos normalizados. No se han inventado salarios municipales: esfuerzo y
asequibilidad personal siguen nulos. El salario regional OVJ de 2024 se utiliza
solo en el análisis EMAL de ese mismo año, no para estos anuncios de 2026.

Transporte, consultas y atomicidad

HDFS conserva y recupera idénticos 31 archivos por carga: originales oficiales,
HTML/robots, sus metadatos y derivados. Kafka confirma 638 mensajes y MongoDB
persiste 638 registros: 621 promociones (215 Visesa y 406 Etxebide) y 17 anuncios.
Los dos rechazos originales de Abanto se producen en la preparación oficial;
los 638 aceptados no producen nuevos rechazos de transporte.

Muestra de anuncios y conclusiones (continuación)

La reentrega de los 17 anuncios mantiene 638 documentos, con cero duplicados
nuevos. También se ha repetido el recorrido completo en la misma base.
Las cinco vistas municipales usan su propio transporte y colección.

Las consultas actuales excluyen promociones y simulaciones. Resultados reales:

Consulta	Resultados	Interpretación
Anuncios de alquiler de Bilbao	6	Identidades observadas con precio positivo
Precio hasta 750 €/mes	1	Umbral analítico, no solvencia ni derecho a ayuda
Bilbao, hasta 2.000 €/mes y desde 55 m²	4	Filtros combinados sobre la muestra
Hasta 10 anuncios con al menos dos habitaciones	10	Campo anidado publicado, sin exigir ascensor desconocido
Señal temporal en array de condiciones	3	Filtrado de una señal textual publicada, no categoría jurídica
Dos habitaciones y ascensor acreditado en campo	0	Ascensor no se ha extraído a un campo observado
Municipios con Gaztelagun publicado en array	0	Los catálogos utilizados no aportan ese array; no se rellena

Se conservaron los tres pipelines: estadísticas por municipio, cruce territorial
con esfuerzo nulo y `$facet` de distribuciones/top. La colección de anuncios
permanece separada por `tipo_registro`; las vistas EMAL no se tratan como anuncios.

`explain` de la consulta Bilbao/alquiler/precio hasta 2.000 examina 638 documentos
con `COLLSCAN` y 4 con `IXSCAN`; ambos devuelven 4. Ambos tiempos medidos son 0 ms:
la reducción de documentos examinados no permite cuantificar aceleración temporal.
No se eliminaron índices ajenos.

La demostración de atomicidad ejecutó 100 incrementos concurrentes sobre una
copia aislada de un anuncio observado y obtuvo contador 100. Es una prueba del
incremento atómico de un documento, no demanda real ni una transacción global.
No se modificó el contador de la colección original.

Reproducción sin recapturar ni entrenar

Con HDFS, Kafka y MongoDB reales preparados y `MONGO_URI` configurada:

```bash
uv sync --locked --extra dev
uv run --no-sync python scripts/audit_project.py
python -c "import shutil; shutil.copytree('data/capturas_anuncios/2026-11-12',
'data/scraping/reproduccion')"
uv run --no-sync python scripts/run_pipeline.py --snapshot-dir data/capturas_oficiales
--raw-dir data/scraping/reproduccion --output-dir data/bda/con_scraping --db-name
reto1_muestra
uv run --no-sync python scripts/analyze_rental_sample.py --capture-dir
data/capturas_anuncios/2026-11-12 --pipeline-dir data/bda/con_scraping --db-name
reto1_muestra
uv run --no-sync python main.py --subject bda_sbd --visual
```

La copia de trabajo recibe los preparados oficiales; el archivo de entrega conserva
sus bytes. El visor abre los resultados archivados verificados;
`RETO1_RENTALS_DIR=data/analisis_anuncios` permite abrir los recién regenerados.
Ninguna de estas órdenes entrena, selecciona modelos ni reevalúa el test conservado.

Cierre de requisitos docentes

Cierre de los puntos de la revisión docente

MIA: predicción que sí cambia la orientación

R_REFERENCIA, prioridad 65, recomienda BUSCAR_ALTERNATIVAS cuando la renta media
municipal predicha supera el 40% del ingreso declarado en el escenario de pagarla
íntegra. Es una señal interna de orientación territorial, no una evaluación de
solvencia ni un requisito de ayudas. Seguridad, vigencia normativa, esfuerzo
real crítico y revisión documental conservan prioridad mayor.

El esfuerzo declarado sigue usando contrato y deudas. El esfuerzo de referencia
usa la predicción municipal y se muestra por separado. El motor tiene diez reglas
y mantiene ocho ramas n8n. La ficha local conserva regla y referencia estimada;
no se transmiten salario, nombre, dirección ni perfil personal.

Prueba contrafactual inventada: ingreso 1.500 €, contrato 500 €, documentación
completa. Predicción de prueba 600 € → CONSULTAR_GAZTELAGUN; predicción 1.600 € →
BUSCAR_ALTERNATIVAS por R_REFERENCIA. Esfuerzo declarado 33,33% en ambos casos;
esfuerzo de referencia 40% / 106,67%. No se ejecuta el modelo para esta prueba.
La prueba AppTest con el modelo conservado comprueba además el caso completo
hasta una ficha n8n real, con consentimiento y repetición sin duplicados.

BDA: API que aporta precios monetarios

Se incorpora el servicio JSON que utiliza el visor público oficial EMAL:
https://eremutentsionatua.euskadi.eus/ad23aConsultaWar/consulta/emal

Su uso se identifica en el visor público y su JavaScript público:
https://eremutentsionatua.euskadi.eus/webzte00-municipios/es/ad23aConsultaWar/emal?locale=es

GET con ambito=4 (municipio), tipoDatoContratos=4 (firmados por trimestre),
tipoContrato=1 (vivienda habitual), tipoHueco=0 (edificio colectivo), y códigos de
provincia/municipio publicados. No necesita autenticación. No es una conversión
local del Excel a JSON ni se utilizó un servicio privado.

Se conserva la respuesta original de cada capital, con URL completa, filtros,
fecha, HTTP, tamaño y SHA-256, junto con la página que documenta el visor, en
`data/capturas_api_rentas/2026-11-12/`. Los normalizados seleccionan 2024 T4:

Municipio	Renta mensual API	Contratos
Vitoria-Gasteiz	798,6 €	549
Donostia-San Sebastián	1.147,5 €	516
Bilbao	930,3 €	767

La API redondea importes a una décima; el Excel conservado publica más decimales.
No se reemplazan sus bytes ni los 768 datos de entrenamiento/validación/test.
Son tres estadísticas de renta municipal; no son anuncios ni promociones.
Se transportan por HDFS → recuperación idéntica → Kafka → MongoDB `rentas_api`.
La reentrega mantiene tres documentos. Los 638 anuncios/promociones previos
mantienen su recuento y colecciones; no se suman ambas unidades.

Reproducción sin recapturar ni entrenar, con servicios reales disponibles:

```bash
python scripts/transport_rent_api.py --input-dir data/capturas_api_rentas/2026-11-12
--report data/bda/api-rentas.json
```

Una nueva captura solo usa un directorio nuevo:

```bash
```

Cierre de requisitos docentes (continuación)

```
python scripts/capture_emal_api.py --output-dir data/capturas_api_rentas/otra_fecha
```

SAA y PIA: contenido obligatorio visible

Los README principal y del bloque IA incorporan el problema cuantificado y su usuario, el esquema del recorrido, la tabla de nueve modelos con métricas de train/validación y tiempos históricos, dos gráficos EDA comentados, captura histórica de MLflow y gráfico de la única evaluación final. Se explican IA débil frente a fuerte y aprendizaje supervisado frente a no supervisado.

Los ejercicios de redes usan Keras y el proyecto MLPRegressor; Kafka y HDFS usan APIs diferentes de las prácticas. Se explica la equivalencia sin decir que son la misma API. TimeSeriesSplit está mencionado en el PDF de validación de clase. SHAP y la comparación auxiliar SMOTE amplían los ejercicios aportados, pero no se garantiza el criterio de novedad de todo el curso. El resultado conservado SMOTE no mejora F1; no se reabre la selección buscando otro resultado.

Defensa

El guion de 15 minutos y las preguntas se actualizan en DEFENSA.md. Los recorridos se comprueban con software y casos de demostración identificados. El ensayo oral cronometrado de la pareja y las tres respuestas individuales de cada alumno no pueden acreditarse con pruebas automatizadas y permanecen a cargo del alumnado.

Validación local del cierre: 168 pruebas, cinco avisos conocidos, 12 integraciones reales sin omisiones, dos transportes de API y ocho ramas n8n. El caso de prueba AppTest del modelo conservado produjo R_REFERENCIA y una ficha real idempotente. Los XML y JSON están en docs/evidencias_cierre_rubrica_2026-11-12/.

Reglas y automatización local

Prioridad descendente; primera regla aplicable. Seguridad: datos inválidos o fuera de cobertura → revisión. Después se revisan vigencia normativa, esfuerzo crítico, requisitos, documentación, alternativas, gastos de entrada y consulta informativa. Fallback: información general.

Streamlit exige consentimiento y envía solo UUID, regla, decisión y referencia estimada. n8n separa ocho ramas de acción; el receptor local persiste una ficha por UUID. Repetir el mismo evento no duplica; reutilizar UUID con contenido diferente se rechaza. Solo se considera éxito con confirmación de persistencia. No se envían mensajes reales.

Las reglas normativas deben contrastarse con la convocatoria aplicable; la referencia no es una resolución administrativa ni la renta media una oferta disponible.

Reproducción y defensa

```
# Defensa de 15 minutos

Ensayar con los artefactos conservados; no entrenar ni volver a evaluar test.
No presentar una ejecución histórica como si acabara de realizarse.

| Minutos | Contenido y demostración |
|---|---|
| 0–2 | Problema, usuario y límite. Bilbao 2024T4: renta 930,31 EUR frente a salario
regional OVJ de 1.564 EUR; escenario 59,48 %, no salario individual ni municipal. |
| 2–4 | Fuentes originales y manifiesto, INE y rechazos. Promociones no son ofertas
disponibles. Mostrar HDFS raw/processed y recuperación idéntica. |
| 4–6 | Kafka → MongoDB: lote, procedencia, persistencia antes del offset, recarga sin
duplicados. Distinguir las vistas EMAL de los mensajes HousingAd. |
| 6–8 | Comparar las tres capitales y mostrar tabla, serie y mapa. Seis indicadores; media
territorial simple y cobertura limitada. |
| 8–10 | Datos y pipeline ML, división temporal 601/75/92, comparativa MLflow y MLP_2_capas
frente a persistencia. Explicar la pequeña diferencia. |
| 10–12 | SHAP de validación y test sellado (MAE 28,67 EUR); abrir la app sin recalcular.
Predecir un caso declarado como demostración. |
| 12–14 | Reglas por prioridad, consentimiento, n8n y ficha local. Repetir UUID para mostrar
idempotencia sin enviar comunicaciones reales. |
| 14–15 | Limitaciones, requisitos pendientes y conclusión vinculada a los indicadores.
Mostrar la muestra archivada verificada y explicar su sesgo y mezcla de modalidades. |

## Reproducción

Desde la raíz con Python 3.12: `uv sync --locked --extra dev`.
`python main.py --audit --json` comprueba integridad;
`python main.py --test -q` ejecuta la suite.
`python scripts/analyze_municipal.py` requiere MongoDB real en MONGO_URI;
`--simulate-db` solo es desarrollo identificado.
`python main.py --subject bda_sbd --visual` abre el visor;
`python main.py --subject saa_pia_mia` abre el modelo conservado.
`python main.py --pdf` genera un informe desde artefactos sin reentrenar.

Los servicios HDFS, Kafka, MongoDB y n8n deben arrancarse expresamente.
Ver `.github/workflows/bda-candidate.yml`, `infra/start_hdfs_ci.sh` y
`n8n/compose.yml`. Los XML con skips no acreditan integración real.

## Preguntas individuales para preparar

- ¿Por qué regresión? El objetivo observado es una renta continua agregada; no hay etiquetas
personales de solvencia.
- ¿Dónde se ajustan imputadores y escaladores? En train y dentro de cada fold temporal,
mediante Pipeline.
- ¿Por qué persistencia? Lag1 y objetivo tienen fuerte correlación; un modelo complejo debe
compararse con esa referencia.
- ¿Por qué no usar SMOTE sobre euros? Genera muestras para clases; no corresponde al
objetivo continuo. Se conserva un experimento auxiliar separado.
- ¿Por qué no cambiar a Keras tras ver test? La selección está cerrada. Los ejercicios
muestran otra biblioteca, pero el enunciado no la fija.
- ¿Qué explica SHAP? Contribuciones del modelo en validación, no causalidad ni derechos
administrativos.
- ¿Cuándo se confirma Kafka? Después de persistencia y verificación. Una caída previa
permite reentrega e inserción idempotente.
- ¿Qué protege un índice único? La identidad de cada dato; no convierte catálogos de dos
fuentes en viviendas únicas.
- ¿Qué significa el esfuerzo? Escenario renta/ingreso regional, no proporción de jóvenes
capaces de emanciparse.
- ¿Qué hace n8n? Guarda acciones locales distintas con consentimiento e idempotencia; no
concede ayudas.
- ¿Qué queda sin acreditar? Ensayo humano, actualización de disponibilidad de los anuncios y
```

Reproducción y defensa (continuación)

cualquier plataforma o integración sin evidencia actual. La captura del 12 de noviembre está conservada.

Ensayo técnico del incremento de anuncios

En los minutos 2–4 mostrar `data/capturas_anuncios/manifest.json`, un HTML y su metadato robots; `scripts/audit_project.py` verifica sus bytes. En los minutos 4–6 mostrar 638 mensajes recibidos (621 promociones + 17 anuncios) y recarga sin duplicados. En los minutos 6–8 abrir la sección de anuncios del visor: 6 Bilbao, 6 Donostia y 5 Vitoria. Comparar precio pedido y superficie, sin confundirlo con EMAL. Explicar 3 señales de temporalidad y 2 de empresas, que pueden solaparse.

Preguntas añadidas: ¿17 anuncios son 17 viviendas distintas? No: son identidades de portal; puede haber publicación cruzada. ¿El índice aceleró el tiempo? Se examinaron 638 frente a 4 documentos; ambos tiempos fueron 0 ms y no permiten cuantificar aceleración. ¿Por qué dos consultas devuelven cero? No se ha observado ascensor en los campos normalizados ni programas municipales publicados en esas fuentes; no se rellenan con constantes. ¿El contador prueba demanda real? No: las 100 operaciones se hicieron sobre una copia aislada para demostrar atomicidad.

El guion está preparado y se ha comprobado el recorrido técnico; el ensayo oral cronometrado y las respuestas personales corresponden al alumnado y siguen pendientes.

Recorrido de cierre para ensayar en pareja

- Minutos 0–2 (persona A): explicar el servicio de orientación y el escenario Bilbao 2024 T4, con salario regional. No hablar de concesiones o solvencia.
- 2–4 (A): mostrar respuesta JSON de API EMAL y sus filtros; contraponerla a un anuncio HTML. Señalar municipio/trimestre frente a identidad de portal.
- 4–6 (A): mostrar los informes HDFS/Kafka/MongoDB, recuperación y recarga. Enseñar explain (638 frente a 4 documentos) y atomicidad sobre copia aislada.
- 6–8 (B): comparar tres capitales, separar anuncios 2026 de estadísticas 2024, explicar los indicadores y que una muestra de conveniencia no es un censo.
- 8–10 (B): enseñar el EDA y tabla MLflow; explicar 601/75/92, lag1, preprocesado en train y la pequeña diferencia frente a persistencia. No entrenar.
- 10–12 (B): SHAP de validación, test sellado leído y nuevo caso. Explicar que R_REFERENCIA usa la predicción para orientar, sin sustituir el contrato.
- 12–14 (A): contrastar el caso de prueba 600/1.600, mostrar consentimiento, ficha n8n con regla/referencia y repetir UUID. No enviar mensajes externos.
- 14–15 (ambas personas): límites, diferencias respecto a clase y conclusión.

Este reparto es un guion de ensayo, no un ensayo oral ya realizado. Practicar con cronómetro; al llegar a 15 minutos se corta como indica el enunciado.

Preguntas individuales nuevas, con respuesta y punto del código que mostrar:

1. SAA: ¿por qué no clasificación? La etiqueta observada es renta continua municipal; enseñar TARGET y temporal_split en src/ml_data.py.
2. SAA: ¿qué pasa si KNN memoriza? Su MAE train casi cero contrasta con MAE val 51,52 €; mostrar comparativa conservada y justificar el ganador sin mirar test.
3. SAA: ¿por qué sklearn y no Keras? Misma familia MLP, otra API; mostrar las cuatro configuraciones en regression_models. No se cambia tras abrir test.
4. PIA: ¿qué contiene joblib? Pipeline con preprocesador y regresor; mostrar construcción en src/ml_workflow.py y carga/inferencia de la app sin fit.
5. PIA: ¿cómo se evita repetir test? Mostrar la rama que lee test_final.json y verifica las huellas en evaluate_test_once, sin pulsar una nueva evaluación.
6. PIA: ¿de dónde sale la tabla? Consulta MLflow y CSV conservado; mostrar sus métricas train/validación y tiempo, no generar valores manualmente.
7. MIA: ¿dónde influye la predicción? R_REFERENCIA y esfuerzo_referencia_pct; enseñar el contrafactual de tests/test_business_rules.py.
8. MIA: ¿puede ganar sobre seguridad? No: prioridad 65 frente a 100; mostrar la prueba que devuelve R_SEGURIDAD incluso con una referencia alta.

Reproducción y defensa (continuación)

9. MIA: ¿qué acción se ejecutó? Una ficha local con regla y referencia, no una cita concertada ni una ayuda concedida; mostrar persist_action y el JSON.

Apéndice: agregaciones municipales completas

Código canónico completo; no se recorta para ajustar a una página.

```
"""Análisis descriptivo EMAL + OVJ en MongoDB, separado del entrenamiento.

Unidad: municipio/trimestre, nunca anuncio ni hogar. El esfuerzo es un escenario
con salario autonómico de 2024, no una estimación del salario municipal.
"""
import json
import math
from pathlib import Path
import pandas as pd
from src.ml_data import read_emal
from src.extraction.source_snapshots import verify_snapshot, MunicipalityCatalog,
parse_ovj_salary


def analyze(db, snapshot_dir, year=2024, quarter=4):
    """Carga idempotente por huella y exporta tres agregaciones del mismo periodo."""
    root = Path(snapshot_dir)
    sources = verify_snapshot(root)
    frame, metadata = read_emal(root)
    salary_source = sources["ovj_salario_2024.html"]
    salary = parse_ovj_salary((root / salary_source["archivo"]).read_bytes(), salary_source)
    if year != int(salary["periodo"]) or quarter not in (1, 2, 3, 4):
        raise ValueError("Comparar con el salario OVJ del mismo año: 2024, trimestre 1 a 4")
    snapshot = sources["emal_municipios.xlsx"]["sha256"]
    context_id = salary_source["sha256"]
    period = year * 4 + quarter - 1
    if frame.loc[frame.periodo.eq(period)].empty:
        raise ValueError("El trimestre elegido no tiene observaciones")
    db.contextos_ovj.update_one({"_id": context_id}, {"$set": salary}, upsert=True)
    db.rentas_emal.create_index([("captura", 1), ("codigo_municipio", 1), ("periodo", 1)],
unique=True)
    selected = frame[["codigo_municipio", "municipio", "provincia", "anio", "trimestre",
"periodo",
                      "renta_media_eur", "numero_contratos", "renta_lag4"]]
    for row in selected.astype(object).where(selected.notna(), None).to_dict("records"):
        identity = {"captura": snapshot, "codigo_municipio": row["codigo_municipio"],
"periodo": row["periodo"]}
        db.rentas_emal.update_one(identity, {"$set": {**row, "contexto_id": context_id,
"fuente": metadata["fuente"]}}, upsert=True)
    match = {"captura": snapshot, "periodo": period}
    comparison = list(db.rentas_emal.aggregate([
        {"$match": match},
        {"$lookup": {"from": "contextos_ovj", "localField": "contexto_id", "foreignField":
"_id", "as": "contexto"}},
        {"$unwind": "$contexto"},
        {"$project": {"_id": 0, "codigo_municipio": 1, "municipio": 1, "provincia": 1,
"anio": 1, "trimestre": 1,
            "renta_media_eur": 1, "numero_contratos": 1,
            "renta_anual_equivalente_eur": {"$multiply": ["$renta_media_eur", 12]},
            "variacion_interanual_pct": {"$cond": [{"$gt": ["$renta_lag4", 0]},
                {"$multiply": [{"$subtract": [{"$divide": ["$renta_media_eur",
"$renta_lag4"]}, 1]}, 100]}, None]},
            "salario_referencia_autonomico_eur": "$contexto.valor",
            "esfuerzo_escenario_pct": {"$multiply": [{"$divide": ["$renta_media_eur",
"$contexto.valor"]}, 100]},
            "diferencia_umbral_30_eur": {"$subtract": ["$renta_media_eur", {"$multiply":
["$contexto.valor", 0.3]}]},
        }},
        {"$sort": {"renta_media_eur": 1, "codigo_municipio": 1}},
    ]))
```

Apéndice: agregaciones municipales completas (continuación)

```
    # Media simple entre municipios: no confundirla con renta de todos los contratos.
    territory = list(db.rentas_emal.aggregate([
        {"$match": match}, {"$group": {"_id": "$provincia", "municipios": {"$sum": 1},
            "media_simple_rentas_municipales_eur": {"$avg": "$renta_media_eur"},
            "contratos_publicados": {"$sum": "$numero_contratos"}}},
        {"$sort": {"_id": 1}},
    ]))
    series = list(db.rentas_emal.aggregate([
        {"$match": {"captura": snapshot, "anio": {"$lte": year}}},
        {"$project": {"_id": 0, "codigo_municipio": 1, "municipio": 1, "anio": 1,
"trimestre": 1,
                      "periodo": 1, "renta_media_eur": 1, "numero_contratos": 1}},
        {"$sort": {"periodo": 1, "codigo_municipio": 1}},
    ]))
    catalog = MunicipalityCatalog.from_snapshot(root, sources)
    points = []
    for row in json.loads((root / "visesa.json").read_text(encoding="utf-8"))["rows"]:
        code = catalog.resolve(row.get("poblacion", ""))
        try:
            lat, lon = float(row["latitud"]), float(row["longitud"])
        except (TypeError, ValueError, KeyError):
            continue
        if code and math.isfinite(lat) and math.isfinite(lon) and 42 <= lat <= 44 and -4 <=
lon <= -1:
            points.append({"id_promocion": str(row.get("idPromocion")), "codigo_municipio":
code,
                           "municipio": catalog.records[code]["municipio"], "lat": lat,
"lon": lon})
    if len(comparison) < 3:
        raise ValueError("La comparación necesita al menos tres municipios")
    return {"comparativa_municipal": comparison, "resumen_territorial": territory,
            "serie_trimestral": series, "mapa_promociones": points,
            "metadatos": {"anio": year, "trimestre": quarter, "unidad":
"municipio/trimestre",
                "fuentes": {name: sources[name] for name in ("emal_municipios.xlsx",
"ovj_salario_2024.html", "visesa.json", "ine_2026.xlsx")},
                "salario": salary,
                "limites": ["El esfuerzo es un escenario con salario autonómico, no una
medida de cada hogar.",
                    "La renta anual equivalente multiplica por 12 la renta mensual: no suma
contratos anuales.",
                    "El umbral del 30% es analítico, no un requisito legal de subvención.",
                    "El mapa localiza promociones Visesa; no representa ofertas disponibles
ni centros municipales.",
                    "La media territorial es simple entre los municipios cubiertos; no
representa todos los contratos.",
                    metadata["advertencia"]]}}
```

Apéndice: agregaciones de anuncios completas

```
"""Indicadores calculados mediante MongoDB. Datos demo excluidos por defecto."""
from typing import Any, Dict, List


class MongoHousingAggregations:
    def __init__(self, db, include_synthetic: bool = False):
        self.db = db
        self.viviendas = db['viviendas']
        self.municipios = db['municipios']
        self.include_synthetic = include_synthetic

    def _filtro(self, alquiler=True):
        result = {'precio': {'$type': 'number', '$gt': 0}, 'codigo_municipio': {'$type':
'string', '$regex': r'^[0-9]{5}$'}}
        result['tipo_registro'] = {'$ne': 'promocion'}
        if alquiler:
            result['tipo_operacion'] = 'alquiler'
        if not self.include_synthetic:
            result['es_sintetico'] = False
        return result

    def pipeline_1_estadisticas_alquiler_por_municipio(self) -> List[Dict[str, Any]]:
        """Une variantes bilingues mediante INE; separa compra de alquiler."""
        return list(self.viviendas.aggregate([
            {'$match': self._filtro()},
            {'$group': {'_id': '$codigo_municipio', 'municipio': {'$first': '$municipio'},
                        'total_anuncios': {'$sum': 1}, 'precio_medio': {'$avg': '$precio'},
                        'precio_minimo': {'$min': '$precio'}, 'precio_maximo': {'$max':
'$precio'},
                        'superficie_media': {'$avg': '$superficie_m2'}, 'precio_m2_medio':
{'$avg': '$precio_m2'}}},
            {'$project': {'_id': 0, 'codigo_municipio': '$_id', 'municipio': 1,
'total_anuncios': 1,
                          'precio_medio': 1, 'precio_minimo': 1, 'precio_maximo': 1,
'superficie_media': 1, 'precio_m2_medio': 1}},
            {'$sort': {'precio_medio': 1, 'codigo_municipio': 1}},
        ]))

    def _pipeline_esfuerzo(self):
        pipeline = [
            {'$match': self._filtro()},
            {'$lookup': {'from': 'municipios', 'localField': 'codigo_municipio',
'foreignField': 'codigo_municipio', 'as': 'datos_municipio'}},
            {'$unwind': {'path': '$datos_municipio', 'preserveNullAndEmptyArrays': True}},
        ]
        salary = {'$ifNull': ['$datos_municipio.economia_laboral.salario_medio_joven_mes',
'$datos_municipio.salario_medio_joven_mes']}
        if not self.include_synthetic:
            salary = {'$cond': [{'$eq': ['$datos_municipio.es_sintetico', False]}, salary,
None]}
        pipeline += [
            {'$set': {'salario_medio_joven': salary}},
            {'$set': {'tasa_esfuerzo_pct': {'$cond': [
                {'$and': [{'$isNumber': '$salario_medio_joven'}, {'$gt':
['$salario_medio_joven', 0]}]},
                {'$multiply': [{'$divide': ['$precio', '$salario_medio_joven']}, 100]},
None,
            ]}}},
            {'$project': {'_id': 0, 'id_origen': 1, 'fuente': 1, 'titulo': 1, 'municipio':
1, 'codigo_municipio': 1,
                          'precio': 1, 'precio_m2': 1, 'es_protegida': 1,
'salario_medio_joven': 1, 'tasa_esfuerzo_pct': 1,
```

Apéndice: agregaciones de anuncios completas (continuación)

```
                        'paro_juvenil_pct': {'$ifNull':
['$datos_municipio.economia_laboral.paro_juvenil_pct',
'$datos_municipio.paro_juvenil_pct']},
                        'procedencia_salario': '$datos_municipio.procedencia', 'url': 1,
'es_sintetico': 1}},
        ]
        return pipeline

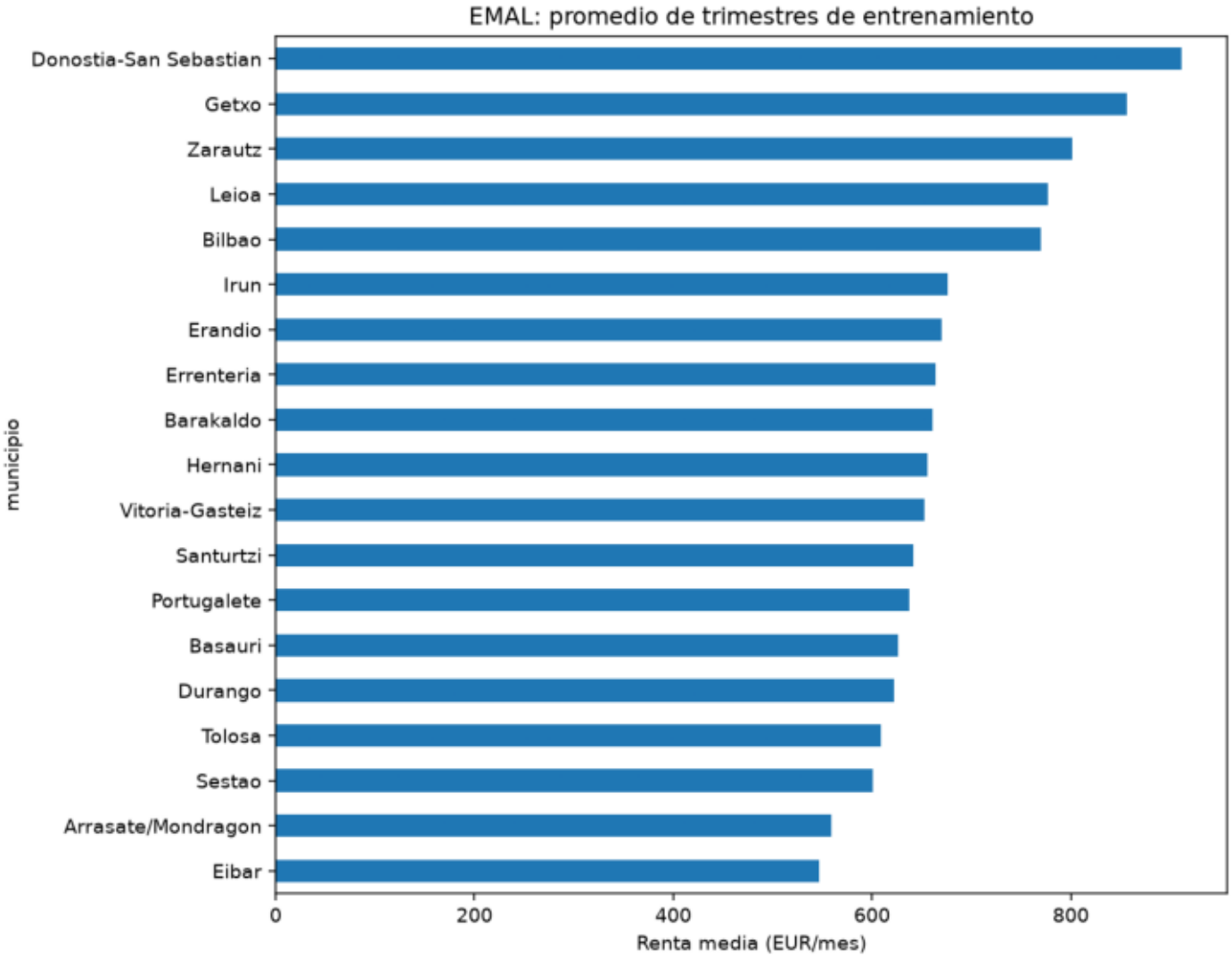
    def pipeline_2_cruce_esfuerzo_salarial(self) -> List[Dict[str, Any]]:
        """Cruce por INE. Sin salario medido no existe porcentaje de esfuerzo."""
        return list(self.viviendas.aggregate(self._pipeline_esfuerzo() + [{'$sort':
{'precio': 1, 'id_origen': 1}}]))

    def pipeline_2_cruce_esfuerzo(self):
        """Alias documentado en el Documento Maestro."""
        return self.pipeline_2_cruce_esfuerzo_salarial()

    def pipeline_3_facet_distribucion_y_top(self) -> Dict[str, Any]:
        """Tres conjuntos visualizables; medias separadas por tipo de operacion."""
        rows = list(self.viviendas.aggregate([
            {'$match': self._filtro(alquiler=False)},
            {'$facet': {
                'distribucion_accesibilidad': [
                    {'$match': {'tipo_operacion': 'alquiler'}},
                    {'$group': {'_id': '$categoria_accesibilidad', 'total': {'$sum': 1}}},
{'$sort': {'_id': 1}},
                ],
                'top_asequibles': [
                    {'$match': {'tipo_operacion': 'alquiler'}}, {'$sort': {'precio': 1,
'id_origen': 1}}, {'$limit': 5},
                    {'$project': {'_id': 0, 'titulo': 1, 'municipio': 1, 'codigo_municipio':
1, 'precio': 1, 'es_protegida': 1}},
                ],
                'desglose_por_tipo': [{'$group': {'_id': '$tipo_operacion', 'total':
{'$sum': 1}, 'precio_promedio': {'$avg': '$precio'}}}],
            }},
        ]))
        return rows[0] if rows else {}

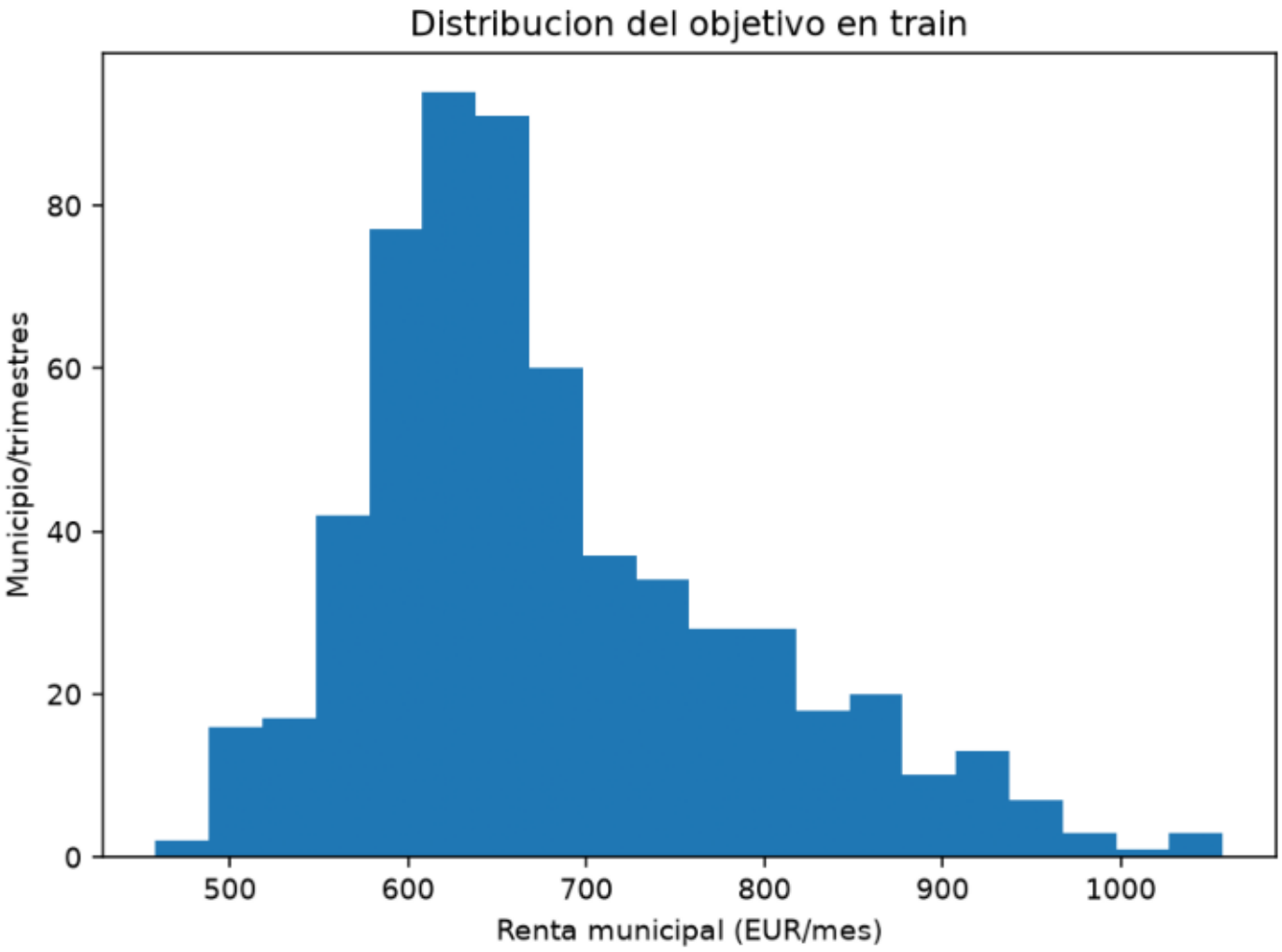
    def indicadores_municipales(self):
        """Cinco indicadores y cobertura: oferta, precio, EUR/m2, esfuerzo y
asequibilidad."""
        valid = {'$isNumber': '$tasa_esfuerzo_pct'}
        pipeline = self._pipeline_esfuerzo() + [
            {'$group': {'_id': '$codigo_municipio', 'municipio': {'$first': '$municipio'},
                        'total_anuncios': {'$sum': 1}, 'precio_medio': {'$avg': '$precio'},
                        'precio_m2_medio': {'$avg': '$precio_m2'}, 'esfuerzo_medio_pct':
{'$avg': '$tasa_esfuerzo_pct'},
                        'anuncios_con_salario': {'$sum': {'$cond': [valid, 1, 0]}},
                        'asequibles_30_pct': {'$sum': {'$cond': [{'$and': [valid, {'$lte':
['$tasa_esfuerzo_pct', 30]}]}, 1, 0]}}}},
            {'$project': {'_id': 0, 'codigo_municipio': '$_id', 'municipio': 1,
'total_anuncios': 1, 'precio_medio': 1,
                          'precio_m2_medio': 1, 'esfuerzo_medio_pct': 1,
'anuncios_con_salario': 1,
                          'porcentaje_asequibles': {'$cond': [{'$gt':
['$anuncios_con_salario', 0]},
                              {'$multiply': [{'$divide': ['$asequibles_30_pct',
'$anuncios_con_salario']}, 100]}, None]}}},
            {'$sort': {'codigo_municipio': 1}},
        ]
        return list(self.viviendas.aggregate(pipeline))
```

01_barras.png



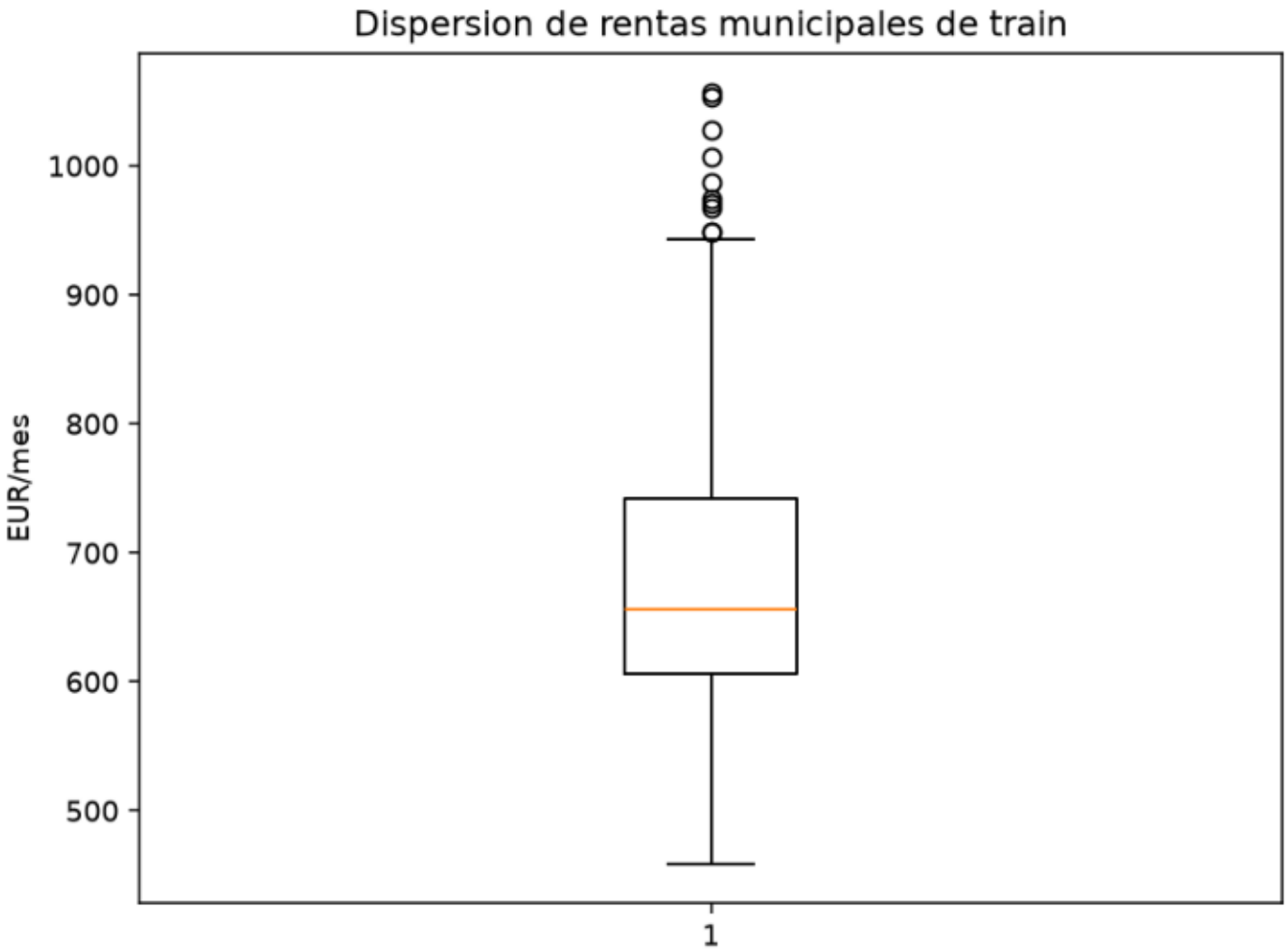
Donostia-San Sebastian presenta la mayor media de train (911.58 EUR). Es una media de agregados trimestrales, no ponderada por contratos.

02_histograma.png



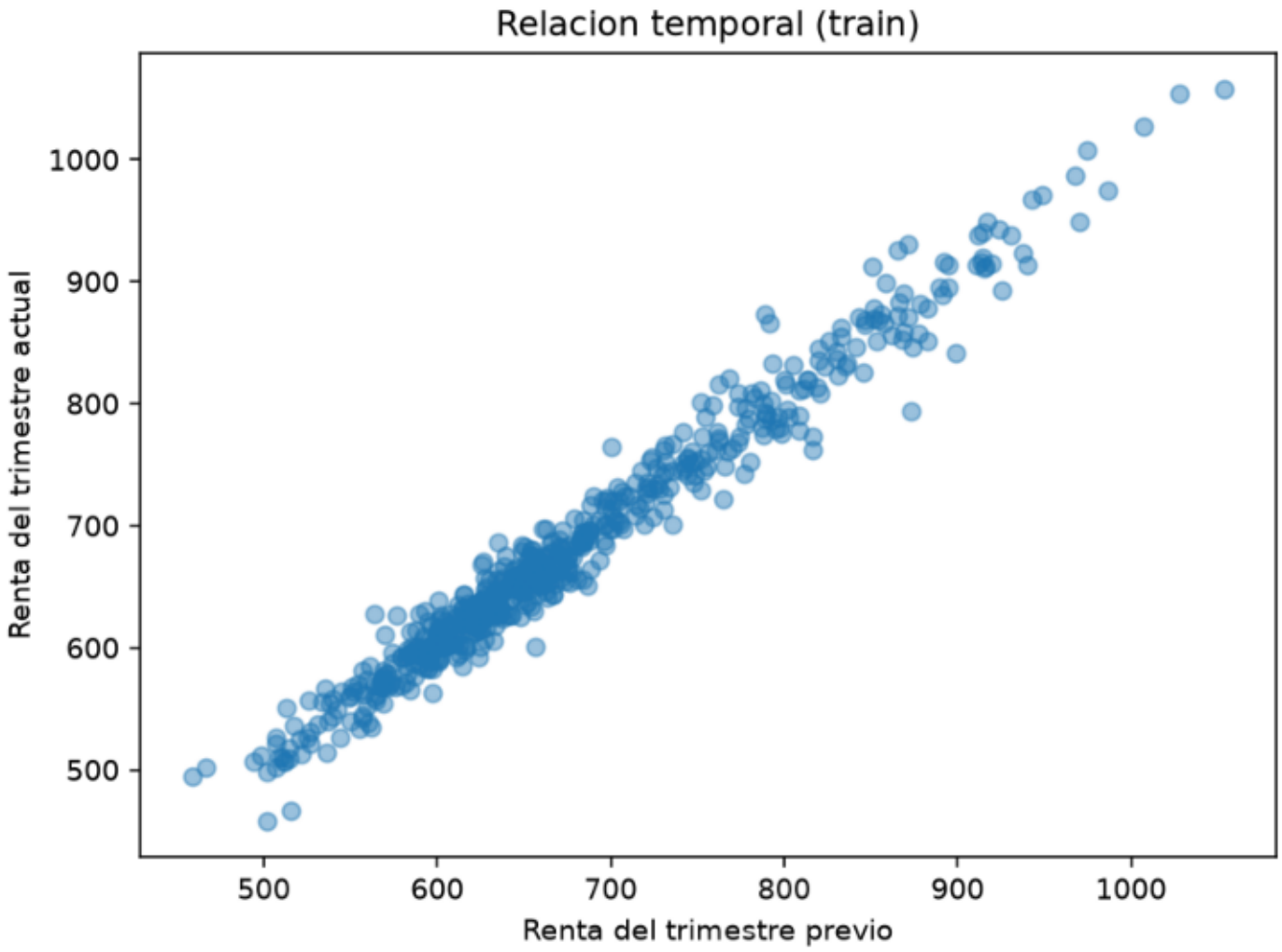
La mediana de train es 656.00 EUR. El objetivo es una renta municipal observada, no la viabilidad de una persona.

03_boxplot.png



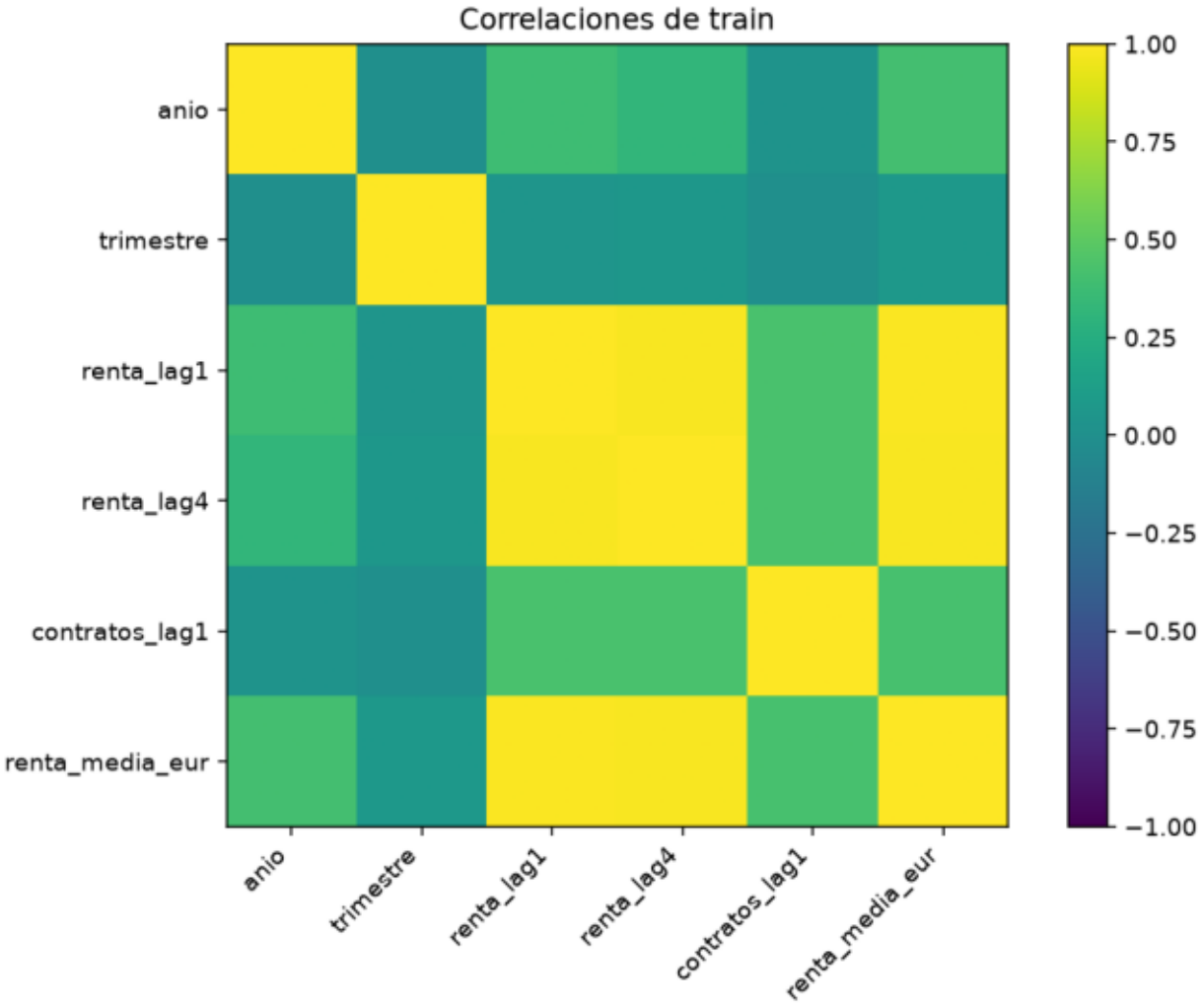
IQR = 136.13 EUR. No se eliminan precios altos solo por ser extremos: pueden reflejar diferencias territoriales reales.

04_dispersion.png



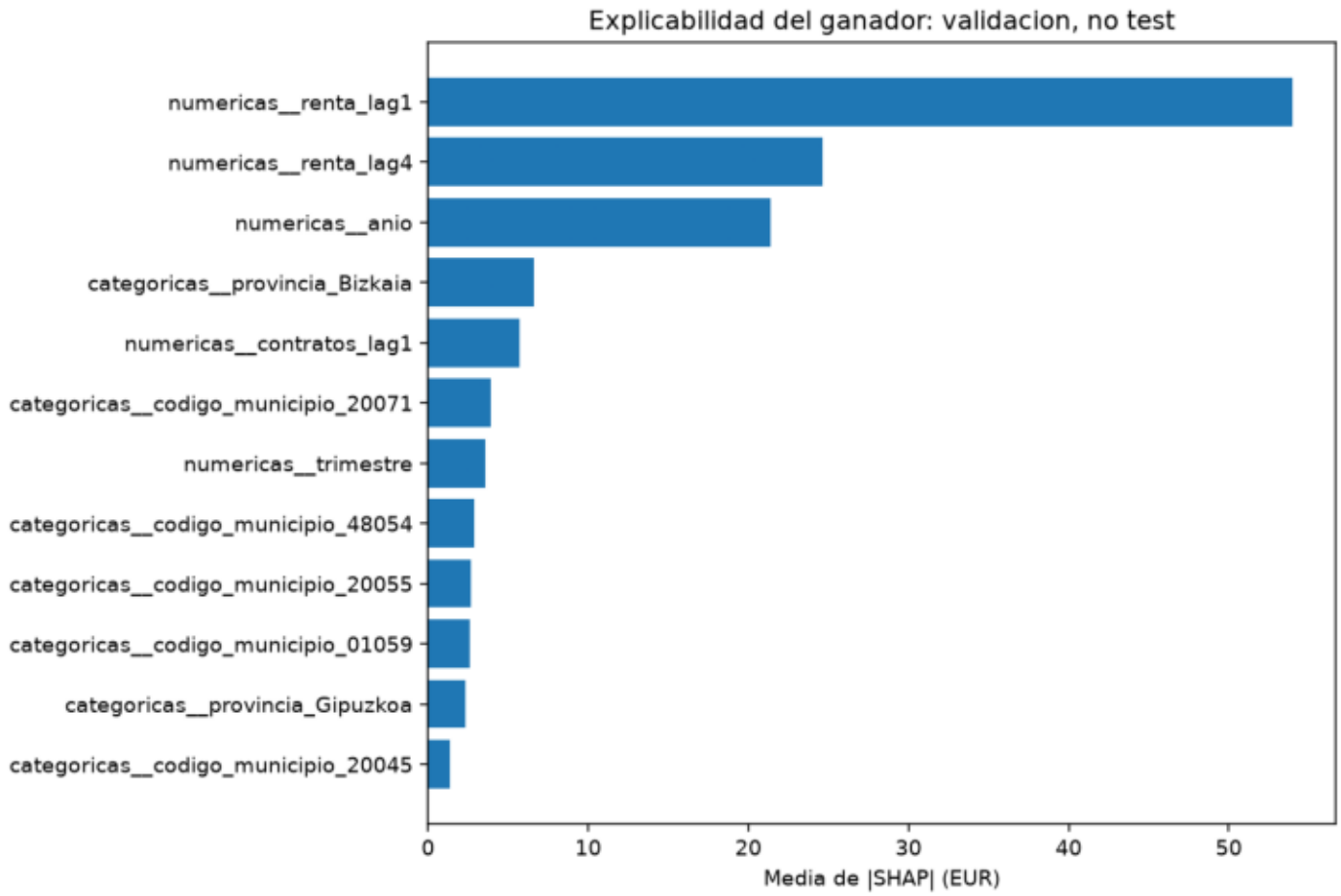
Correlacion lag1/objetivo = 0.985. La persistencia debe ser un baseline; correlacion no demuestra causalidad.

05_correlacion.png



Los retardos recogen persistencia. Ni la renta actual, ni su precio/m2, ni una categoria derivada del objetivo se incluyen entre las entradas.

shap_validacion.png



SHAP de validación; no causal, fondo de train.

test_real_predicho.png

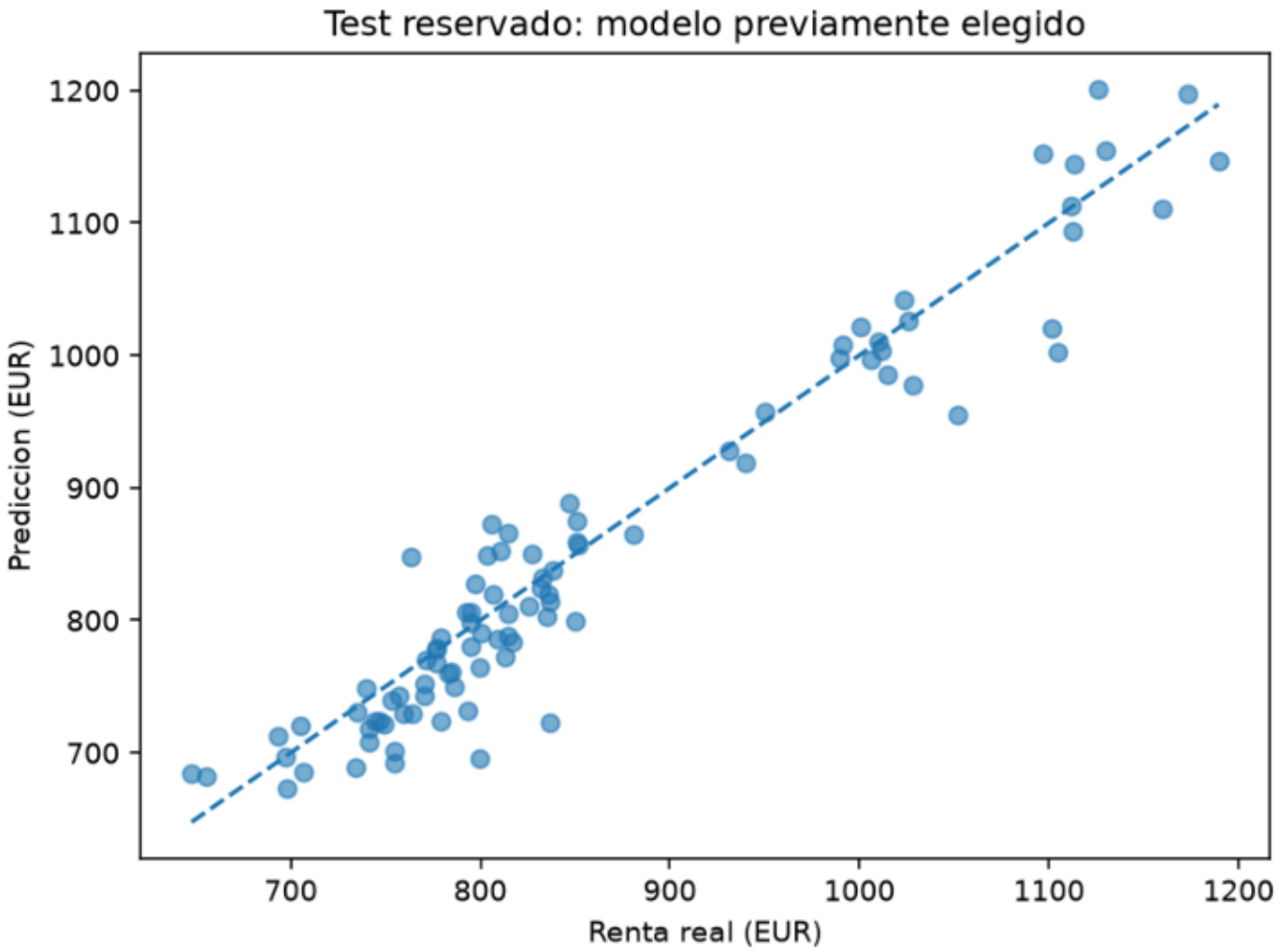


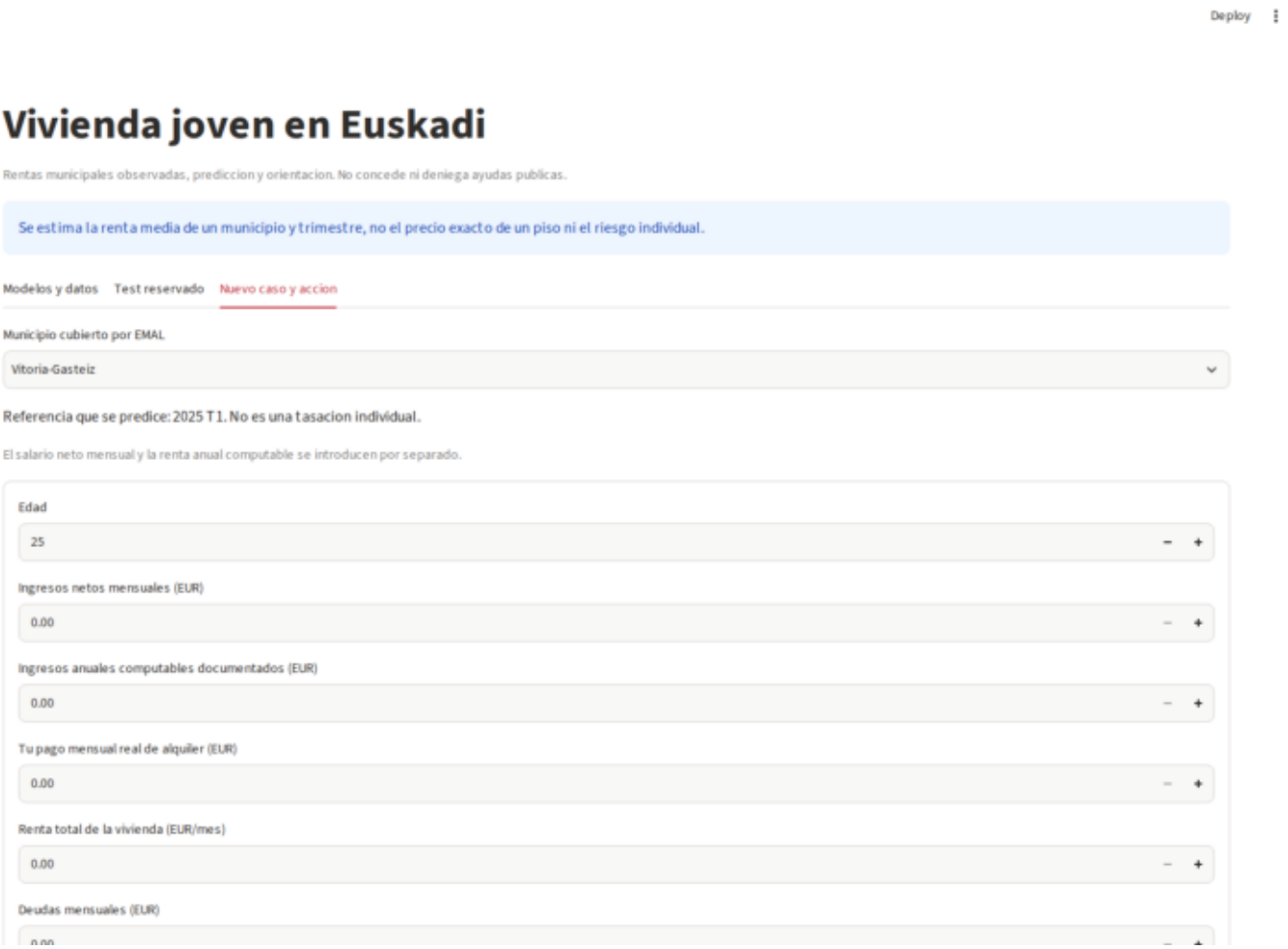
Figura conservada de la única evaluación final; no se recalcula.

mlflow-experimentos.png



Captura histórica conservada del bloque ML; no nueva ejecución.

streamlit-formulario.png



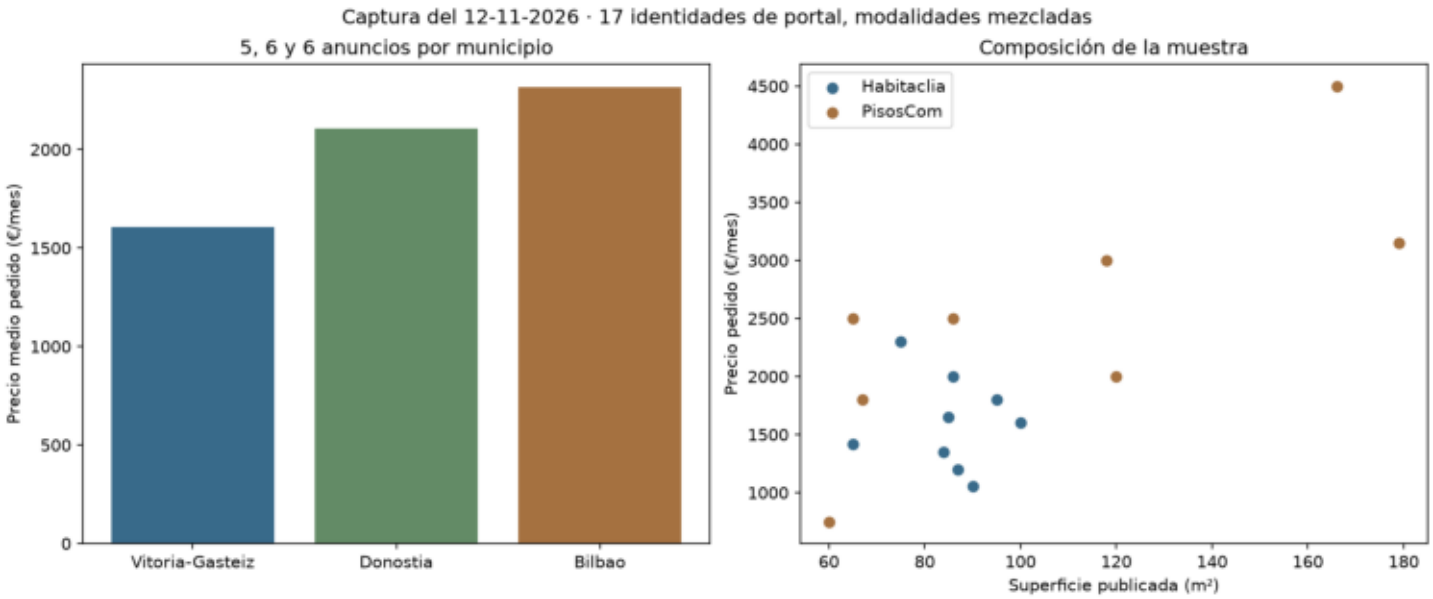
Captura histórica conservada del bloque ML; no nueva ejecución.

streamlit-modelos.png



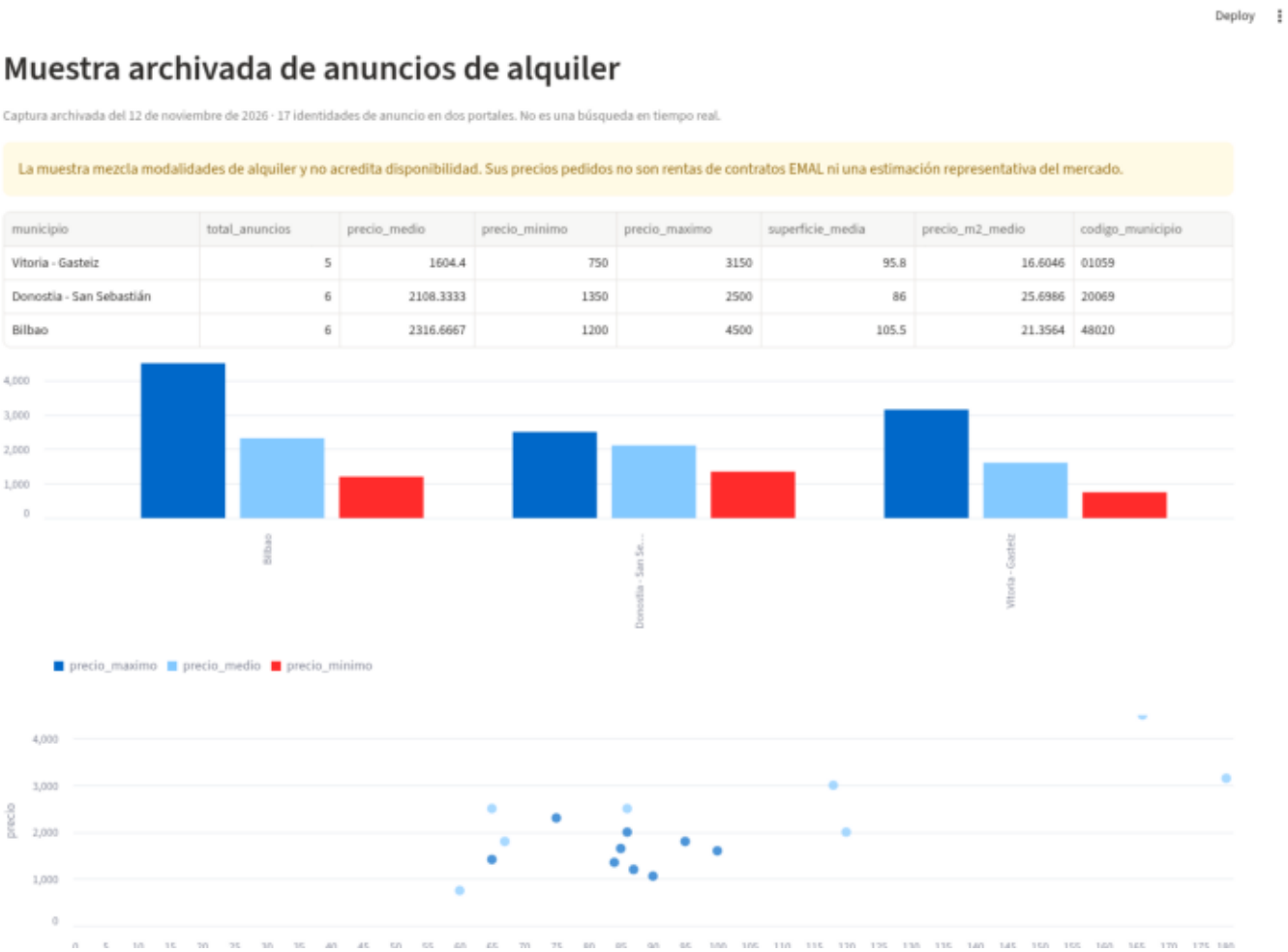
Captura histórica conservada del bloque ML; no nueva ejecución.

muestra_anuncios.png



17 identidades de portal del 12 de noviembre; mezcla de modalidades y muestra de conveniencia.

territorio.png



Captura de Chromium del visor con EMAL y la muestra archivada; no búsqueda en tiempo real.

modelo.png



Aplicación abierta con el test conservado; no se reevalúa el holdout.